```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Globalization;

namespace EcommerceConsoleApp
{
    // Product model
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public decimal Price { get; set; }
    }

    // Cart item
    public class CartItem
    {
        public Product Product { get; set; } = null!;
        public int Quantity { get; set; }
        public decimal TotalPrice => Product.Price * Quantity;
    }

    // Order
    public class Order
    {
        public List<CartItem> Items { get; set; } = new List<CartItem>();
        public decimal Total => Items.Sum(i => i.TotalPrice);
    }

    // Repository
    public class ProductRepository
    {
        private readonly List<Product> _products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Price = 18000 },
            new Product { Id = 2, Name = "Phone", Price = 9000 },
            new Product { Id = 3, Name = "Tablet", Price = 5500 }
        };

        public IEnumerable<Product> GetAll() => _products;
        public Product? GetById(int id) => _products.FirstOrDefault(p => p.Id
== id);
    }

    // Order service
    public class OrderService
    {
        private readonly List<Order> _orders = new List<Order>();

        public void PlaceOrder(Order order)
        {
            _orders.Add(order);
            Console.WriteLine($"Order placed successfully! Total:
{order.Total:C}");
        }

        public IEnumerable<Order> GetAllOrders() => _orders;
    }

    class Program
    {
        static void Main()
        {
```

```
            // Set currency to South African Rand
            CultureInfo.DefaultThreadCurrentCulture = new CultureInfo("en-ZA");

            var repo = new ProductRepository();
            var orderService = new OrderService();
            var cart = new List<CartItem>();

            Console.WriteLine("=== Welcome to the Console E-Commerce App ===");

            while (true)
            {
                Console.WriteLine("\n1. List Products\n2. Add Product to
Cart\n3. View Cart\n4. Checkout\n5. Exit");
                Console.Write("Select an option: ");
                var input = Console.ReadLine();

                switch (input)
                {
                    case "1":
                        Console.WriteLine("\nAvailable Products:");
                        foreach (var p in repo.GetAll())
                        {
                            Console.WriteLine($"{p.Id}. {p.Name} -
{p.Price:C}");
                        }
                        break;

                    case "2":
                        Console.Write("Enter Product Id to add: ");
                        if (int.TryParse(Console.ReadLine(), out int prodId))
                        {
                            var product = repo.GetById(prodId);
                            if (product != null)
                            {
                                var cartItem = cart.FirstOrDefault(c =>
c.Product.Id == prodId);
                                if (cartItem != null)
                                    cartItem.Quantity++;
                                else
                                    cart.Add(new CartItem { Product = product,
Quantity = 1 });

                                Console.WriteLine($"{product.Name} added to
cart.");
                            }
                            else
                                Console.WriteLine("Product not found.");
                        }
                        break;

                    case "3":
                        Console.WriteLine("\nCart Items:");
                        if (!cart.Any())
                        {
                            Console.WriteLine("Cart is empty.");
                        }
                        else
                        {
                            foreach (var item in cart)
                            {
                                Console.WriteLine($"{item.Product.Name}
x{item.Quantity} = {item.TotalPrice:C}");
                            }
```

```
                                Console.WriteLine($"Total: {cart.Sum(c =>
c.TotalPrice):C}");
                            }
                            break;

                        case "4":
                            if (!cart.Any())
                            {
                                Console.WriteLine("Cart is empty. Add items
first.");
                            }
                            else
                            {
                                var order = new Order { Items = new
List<CartItem>(cart) };
                                orderService.PlaceOrder(order);
                                cart.Clear();
                            }
                            break;

                        case "5":
                            Console.WriteLine("Thank you for using the app.
Goodbye!");
                            return;

                        default:
                            Console.WriteLine("Invalid option.");
                            break;
                    }
                }
            }
        }
}
```